DressMe

Script du présentateur

8 agents LangGraph, dont un orchestrateur conversationnel

4 services FastAPI, 1 RAG cité, protocole A2A 0.3.0

Plan de contrôle / plan de données, entrées-sorties, questions du jury

---

Présentation technique - 18 minutes - 15 slides

Projet de fin d'études - React 19, FastAPI, LangGraph, OpenAI, Supabase

Comment utiliser ce script

Durée totale : 18 minutes — 14 minutes d'exposé et 4 minutes de démonstration.

Slides : 15. Une idée par slide. La slide est la preuve ; vous êtes l'argument. Ne lisez jamais une slide à voix haute.

Repères : MONTRER = ce qui est à l'écran. **DIRE** = le texte parlé, à adapter et non à réciter. **ENCHAÎNER** = la phrase de transition vers la slide suivante.

Règle d'or : chaque affirmation technique doit finir sur un bénéfice ou un compromis. Un jury retient le « pourquoi », pas le « quoi ».

Secours : les PDF de présentation (`DressMe_Presentation_FR.pdf`) contiennent les figures et le détail complet si une question va plus loin.

Précision sur les modèles — à dire si on vous le demande. Les huit agents ne tournent pas tous sur le même modèle. Les agents BO1 et l'agent Trip utilisent **GPT-4o** par défaut (`LLM_MODEL=gpt-4o`). L'orchestrateur autonome 8070 et BO4 utilisent **qwen2.5:3b** en local via Ollama. L'agent Purchase utilise un **Qwen2.5-3B fine-tuné + LoRA**, GPT-4o ne fait que réécrire le verdict, et XGBoost sert de repli déterministe. Présentez cela comme un choix assumé : le bon modèle pour la bonne tâche, pas un modèle unique partout.

Partie 1 — L'exposé (0:00–14:00)

0:00 — Ouverture (30 s)

MONTRER : la slide de titre avec le sous-titre en une ligne.

DIRE : « DressMe transforme la photo d'un vêtement en garde-robe structurée, puis huit agents LangGraph raisonnent sur de vrais vêtements, la vraie météo et des livres de mode cités pour répondre à cinq questions : quoi porter, quelles tenues composer, quoi emporter, que me manque-t-il, et dois-je acheter ceci ? La règle est simple : le modèle décide et rédige ; le code déterministe vérifie et calcule. »

ENCHAÎNER : « Commençons par pourquoi ce n'est pas une énième galerie de vêtements. »

0:30 — Problème et promesse (45 s)

MONTRER : trois points de douleur : garde-robe invisible, aucune logique de combinaison, valise et achat irrationnels.

DIRE : « Les applications existantes sont des galeries. Elles montrent les vêtements mais ne raisonnent pas dessus. DressMe ajoute trois choses : une garde-robe structurée, un raisonnement ancré et des solveurs déterministes. Et nous faisons trois promesses : ne jamais inventer un vêtement, ne jamais inventer la météo, ne jamais inventer une source. »

ENCHAÎNER : « Voici ce que cela donne à l'écran. »

1:15 — Parcours produit (60 s)

MONTRER : trois captures : réponse de l'AI Stylist avec météo, tenue et sources `[S#]` ; Outfit Builder avec trois tenues ; checklist Help Me Pack.

DIRE : « Chaque bloc affiché est adossé à de vraies données. La réponse du styliste porte la météo et les sources utilisées. Les cartes de tenue n'utilisent que des vêtements possédés par l'utilisateur. La checklist de valise est validée par un solveur, pas écrite librement par le modèle. Le backend renvoie aussi `agents_called`, qui trace l'agent qui a répondu. »

ENCHAÎNER : « Sous l'interface, quatre services déployables indépendamment. »

2:15 — Architecture globale (75 s)

MONTRER : Figure 1 — services et ports.

DIRE : « React 19 appelle BO1 sur 8000 pour la garde-robe, le styliste, la similarité et les tenues, BO3 sur 8001 pour l'analyse des manques, Help Me Pack sur 8020 pour les voyages, et le service RAG sur 8010 pour la connaissance mode. Supabase fournit l'authentification, Postgres et le stockage. OpenAI et Open-Meteo sont les seuls fournisseurs externes. Deux alternatives locales existent : l'orchestrateur autonome 8070 et le styliste BO4 8061 ; elles ne sont pas branchées à React. »

ENCHAÎNER : « L'architecture ne vaut que si les données sont protégées. »

3:30 — Données, authentification et mémoire (45 s)

MONTRER : Figure 2.

DIRE : « Chaque requête porte un JWT Supabase. Une dépendance FastAPI résout l'utilisateur, et chaque requête SQL est filtrée par cet identifiant. Le même jeton est transmis sur les tâches A2A, donc un agent ne peut pas lire la garde-robe d'un autre utilisateur. La mémoire de conversation est un checkpointer LangGraph ; une correction de garde-robe expire au bout de quinze minutes si l'utilisateur ne confirme pas. »

ENCHAÎNER : « La première étape du pipeline, c'est la vision. »

4:15 — Pipeline de vision (45 s)

MONTRER : Figure 3.

DIRE : « YOLO passe d'abord pour la géométrie et la couleur, mesurées sur les pixels. GPT-4o vision passe ensuite pour la sémantique, avec un JSON strict et une taxonomie fixe de vingt classes. Si le modèle est indisponible, un repli déterministe produit quand même un vêtement valide. La raison est simple : le modèle ne doit pas deviner la couleur, et il ne doit pas inventer un vêtement absent de la photo. »

ENCHAÎNER : « Une fois la garde-robe créée, huit agents travaillent dessus. »

5:00 — Les huit agents (60 s)

MONTRER : le tableau des agents : agent, rôle, modèle.

DIRE : « Un orchestrateur et sept spécialistes. L'AI Stylist répond à la question ouverte ; Wardrobe possède les données et les écritures ; Similar décide possédé, proche ou manquant ; Outfit Builder compose ; Trip planifie et prépare la valise ; Gap analyse ; Purchase décide. Chaque agent a un contrat étroit, ses outils et son évaluation. GPT-4o pour les agents BO1 et Trip, la famille GPT-4o pour Gap, Qwen fine-tuné plus LoRA pour la décision d'achat, qwen local pour les alternatives non branchées. »

ENCHAÎNER : « Mais un agent n'est pas un appel de modèle. C'est un graphe. »

6:00 — LangGraph : pourquoi (75 s)

MONTRER : Figure 4, puis le graphe Wardrobe `gate → apply / cancel / assist`.

DIRE : « LangGraph n'est pas décoratif. Il apporte cinq choses qu'une simple fonction n'apporte pas : un état partagé explicite, des arêtes conditionnelles, un checkpointer pour la mémoire de conversation par utilisateur, des interruptions pour l'`input-required`, et un modèle de boucle d'outils réutilisé par tous les agents. Regardez Wardrobe : la porte décide d'appliquer, d'annuler ou d'assister ; c'est une machine à états, pas un if. Regardez Outfit Builder : la validation reboucle vers la composition. Regardez Trip : deux graphes, un conversationnel et un déterministe. »

DIRE : « Ce pour quoi LangGraph n'est pas fait : YOLO, CP-SAT, XGBoost et la recherche RAG. Ce sont des moteurs déterministes et ils restent du code simple. La règle : du graphe là où il y a du contrôle avec état, du code simple là où il y a un algorithme exact. »

ENCHAÎNER : « Maintenant le point qui surprend : l'orchestrateur ne possède pas tous les messages. »

7:15 — Orchestrateur : le rôle limité (75 s)

MONTRER : Figure 14 — plan de contrôle contre plan de données.

DIRE : « L'orchestrateur est le plan de contrôle. Il route chaque tour utilisateur avec GPT-4o en JSON plus des règles françaises et anglaises, il garde l'état de la conversation, il délègue au styliste, à la garde-robe, aux tenues, au voyage ou aux manques via A2A, et il récupère le snapshot de garde-robe pour le transmettre aux agents. C'est l'intention utilisateur. »

DIRE : « Ce n'est pas un bus de messages universel. Les appels de capacité se font sur le plan de données. L'AI Stylist et l'agent Similar appellent l'agent Wardrobe en processus pour une lecture seule. L'agent Trip appelle directement l'agent Gap via A2A avec `check_gaps`. Les routes hors chat utilisent `call_with_wardrobe` pour passer la garde-robe à un agent sans entrer dans le graphe de chat. Si on routait tout cela par l'orchestrateur de chat, il deviendrait un god object, un goulot d'étranglement et un point de défaillance unique. »

ENCHAÎNER : « Alors comment les agents se parlent-ils hors du chat ? Avec A2A. »

8:30 — A2A : pourquoi (90 s)

MONTRER : Figure 13 — cartes A2A et cycle de vie des tâches.

DIRE : « A2A 0.3.0, JSON-RPC 2.0. Chaque agent publie une carte publique sur `/.well-known/agent-card.json`. Les méthodes sont `message/send`, `tasks/get` et `tasks/cancel`. Les états sont submitted, working, input-required, completed, failed, canceled et rejected. Le jeton voyage avec la tâche, et une tâche n'est lisible que par son créateur. »

DIRE : « Pourquoi pas du REST ad hoc ? Parce qu'A2A apporte la découverte, un cycle de vie standard, des erreurs standard et l'interopérabilité. Le même contrat est utilisé par l'orchestrateur, par un script et par un autre agent. Exemple concret : Trip envoie sa capsule à Gap ; si Gap est éteint, le voyage continue avec une erreur propre au lieu de planter. C'est le standard qui travaille vraiment. »

ENCHAÎNER : « Rendons les contrats concrets. »

10:00 — Entrées et sorties (40 s)

MONTRER : le tableau des entrées/sorties sur la slide.

DIRE : « Chaque fonctionnalité a une requête JSON explicite et une réponse explicite. La réponse porte de vrais identifiants de vêtements, `sources` quand la connaissance est utilisée, et `decision_source` quand un verdict est produit. C'est ce qui garde l'interface découplée et les tests honnêtes. »

ENCHAÎNER : « La couche de connaissance est ce qui rend le conseil défendable. »

10:40 — RAG et algorithmes (75 s)

MONTRER : Figure 15 et le tableau des algorithmes.

DIRE : « Trente-huit PDF, 14 574 chunks, des embeddings all-MiniLM-L6-v2, une recherche hybride avec fusion de rangs réciproque, les cinq meilleurs passages. Les réponses citent `[S#]` et refusent quand le contexte est insuffisant. Les chiffres mesurés : Hit@5 91,67 %, MRR 85,42 %, précision du contexte 71,67 %. »

DIRE : « Autour du RAG, des algorithmes déterministes font le travail exact : YOLO pour la détection, OR-Tools CP-SAT pour la valise, XGBoost comme base d'achat, un score de tenue transparent — couleur 0,30, style 0,20, occasion 0,30, saison 0,20 —, FashionCLIP et CLIP pour la similarité visuelle. Le modèle là où il faut du jugement ; l'algorithme là où il faut de l'exactitude. »

ENCHAÎNER : « Un système n'est crédible que s'il échoue proprement. »

11:55 — Qualité, garde-fous, tests (45 s)

MONTRER : le tableau des garde-fous.

DIRE : « Ancrage sur de vrais identifiants, la vraie météo et des sources citées. Validation par le code au-dessus du modèle : règles de tenue, Pydantic plus solveur pour les voyages, verdicts d'achat vérifiés. Dégradation gracieuse partout : si un agent est éteint, l'appelant reçoit une erreur propre ou un repli. Et l'honnêteté : le modèle dit « je ne sais pas » au lieu d'inventer, et nous assumons que la base XGBoost est entraînée sur des données synthétiques. »

DIRE : « Tests : 101 sur l'orchestrateur, 85 sur le backend, 30 sur le frontend, 1 095 sur BO4 avec 25 ignorés, plus des essais de bout en bout. »

ENCHAÎNER : « Aucun système n'est fini. Voici ce qui ne l'est pas. »

12:40 — Limites et feuille de route (45 s)

MONTRER : le tableau des limites.

DIRE : « Le RAG ne connaît pas la garde-robe ; les deux sont volontairement séparés. XGBoost est synthétique tant que nous n'avons pas de vrais retours. La mémoire de Trip est en processus. BO4 et 8070 ne sont pas branchés à React. A2A n'a pas encore de streaming ni de push. Feuille de route : mémoire des préférences par utilisateur, journal des tenues portées, streaming A2A et orchestrateur entièrement A2A. »

ENCHAÎNER : « Je conclus. »

13:25 — Conclusion (30 s)

MONTRER : la phrase de conclusion à l'écran.

DIRE : « DressMe est une garde-robe multi-agents ancrée : huit agents LangGraph, quatre services, un RAG cité, A2A et des solveurs déterministes. L'orchestrateur décide qui répond ; LangGraph décide comment chaque agent répond ; A2A permet aux agents de se consulter ; et le code garantit que la réponse est vraie. »

ENCHAÎNER : « Je vous montre les cinq flux dans le produit, puis je prends vos questions. »

Partie 2 — Les quatre justifications (à mémoriser)

Pourquoi LangGraph ?

Parce que les parties difficiles sont du contrôle avec état : portes de confirmation, boucles de validation, planification de voyage à deux graphes, mémoire de conversation et reprise après `input-required`. LangGraph rend cela explicite et observable. Il n'est volontairement pas utilisé pour YOLO, CP-SAT, XGBoost ou la recherche.

Pourquoi A2A ?

Parce que les agents sont des services, pas des fonctions. A2A apporte la découverte, un cycle de vie de tâche standard, l'authentification et un contrat réutilisable par l'orchestrateur, un script ou un autre agent. Du REST ad hoc marcherait, mais il faudrait réinventer les cartes, les états et les erreurs.

Pourquoi l'orchestrateur ne route-t-il pas tout ?

Parce qu'il est le plan de contrôle, pas le plan de données. Router chaque lecture interne par lui crée un goulot, un god object et un couplage fort. Les appels de capacité sont directs et en lecture seule ; les écritures restent chez l'agent propriétaire.

Pourquoi hybride plutôt qu'un cerveau central ?

Parce que le système a besoin à la fois d'intention globale et d'autonomie locale. L'orchestrateur décide qui répond ; chaque agent décide comment répondre avec ses outils et ses garde-fous. C'est ce qui rend les agents testables et déployables indépendamment.

Partie 3 — Les cinq flux de communication

- **Chat utilisateur.** React appelle `POST /api/assistant/chat` ; l'orchestrateur route, appelle un agent via A2A, et le nœud `respond` écrit la réponse dans le fil checkpointé.
- **Capacité garde-robe.** L'AI Stylist et l'agent Similar appellent `wardrobe_agent.ask()` en processus ; l'appel est en lecture seule.
- **Trip vers Gap.** `check_gaps` envoie la capsule de voyage à l'agent Gap via A2A avec le jeton utilisateur.
- **Trip vers Wardrobe.** L'agent Trip liste les vêtements via A2A, avec la route BO1 en repli.
- **Routes hors chat.** `call_with_wardrobe()` fait le passage Wardrobe vers agent directement, hors de l'orchestrateur de chat.

Dites cette phrase : « L'orchestrateur coordonne l'intention utilisateur ; les agents portent leurs propres capacités. »

Partie 4 — Entrées et sorties (aide-mémoire)

- **AI Stylist** — entrée : `message`, `city`, `country`, `unit` ; sortie : `answer`, `weather`, `outfit`, `why`, `alternatives`, `tips`, `sources`, `items`.
- **Wardrobe Agent** — entrée : `message`, `action` et `confirm` optionnels ; sortie : `reply`, `items`, `filters`, `proposal`.
- **Similar Agent** — entrée : `pieces` via A2A ; sortie : par pièce `status` (owned, close, missing), `match_item`, `missing_spec`.
- **Outfit Builder** — entrée : `occasion`, `season`, `city`, `avoid` ; sortie : `outfits` avec `title`, `items`, `weather_note`, `why`, plus `missing` et `sources`.
- **Trip Agent** — entrée : `message`, `session_id` ; sortie : `message`, `state`, `capsule`, `checklist`, `alternatives`.
- **Gap Agent** — entrée : `message`, `goal` ; sortie : `answer`, `gaps`, `covered`, `products`.
- **Purchase Agent** — entrée : `image` ; sortie : `decision`, `score`, `reasons`, `matching_item_ids`, `decision_source`.
- **Pipeline avatar** — entrée : `photo` ou `outfit_id` ; sortie : `job_id`, `status`, `result_url`.

Partie 5 — Questions du jury

« Pourquoi pas un seul gros agent ? »

Séparation des responsabilités, propriété des données, évaluation indépendante et rayon d'impact plus faible. Wardrobe possède la garde-robe ; Purchase possède le verdict ; Trip possède la valise.

« Pourquoi LangGraph plutôt que des boucles simples ? »

État, arêtes conditionnelles, checkpointing, interruptions et boucle d'outils réutilisable. Sinon, il faudrait réécrire les cinq.

« Pourquoi A2A plutôt que REST ? »

Découverte, cycle de vie de tâche, interopérabilité et un contrat pour tous les clients. REST marcherait, mais il faudrait réinventer la carte, les états et les erreurs.

« Comment empêchez-vous l'hallucination ? »

De vrais identifiants, la vraie météo, des sources citées, une validation par le code, des replis déterministes et un « je ne sais pas » explicite.

« Pourquoi Qwen fine-tuné pour l'achat et GPT-4o ailleurs ? »

La décision d'achat est étroite et répétitive : un petit modèle LoRA est moins cher et plus contrôlable. GPT-4o rédige l'explication. XGBoost est le repli déterministe. Les agents principaux ont besoin de raisonnement large, donc GPT-4o.

« L'orchestrateur est-il un goulot ? »

Pour le chat, il est le point d'entrée par conception ; les appels de capacité le contournent. Chaque service monte en charge horizontalement, et la ressource chère est l'appel LLM, pas le nœud de routage.

« Pourquoi garder BO4 et 8070 s'ils ne sont pas branchés ? »

Ce sont des variantes locales sans clé, gardées pour comparaison. Nous les présentons comme non branchées au lieu de prétendre qu'elles font partie du produit.

« Quelle a été la partie la plus difficile ? »

La continuité `input-required` du voyage et les contraintes CP-SAT ; puis la confirmation d'écriture dans Wardrobe, parce qu'une mauvaise écriture est pire qu'une mauvaise réponse.

« Comment évaluez-vous les agents ? »

Tests unitaires avec faux LLM, RAG et météo ; tests d'intégration ; essais de bout en bout ; métriques RAG ; et une trace `agents_called`.

« Que feriez-vous avec trois mois de plus ? »

Mémoire des préférences, journal des tenues portées, vrais retours d'achat pour XGBoost, streaming A2A et orchestrateur entièrement A2A.

Partie 6 — Script de démonstration (4 minutes)

1. Ajouter trois vêtements par photo ; montrer la fiche générée et la couleur mesurée.
2. Wardrobe : « montre-moi mes vestes pour le travail » ; montrer les filtres choisis par le LLM.
3. AI Stylist : « que porter demain soir pour un dîner à Tunis ? » ; montrer météo, tenue, raisons et sources.
4. Outfit Builder : composer trois tenues, puis mélanger.
5. Help Me Pack : « Paris, du 3 au 7 juin, 8 kg » ; montrer la clarification, puis la checklist.
6. Gap Agent : montrer les manques classés et les produits tunisiens.
7. Purchase : montrer le verdict, les preuves et le repli ; puis `python scripts/a2a_discover.py --ask` pour les cartes et une vraie tâche.

Si un service est éteint, dites quel est le repli au lieu de le cacher. Cela fait partie du récit.

Partie 7 — Conclusion, checklist et version courte

Récapitulatif des temps

- 0:00 — Ouverture — 30 s
- 0:30 — Problème et produit — 1 min 45
- 2:15 — Architecture, données et vision — 2 min 45
- 5:00 — Agents, LangGraph et orchestrateur — 3 min 30
- 8:30 — A2A, entrées-sorties et RAG — 3 min 25
- 11:55 — Qualité, limites et conclusion — 2 min 05
- 14:00 — Démonstration — 4 min
- 18:00 — Fin

Phrase de conclusion

« L'orchestrateur décide qui répond ; LangGraph décide comment chaque agent répond ; A2A permet aux agents de se consulter ; et le code déterministe garantit que la réponse est vraie. »

Checklist avant de passer

- Cinq services démarrés : BO1 8000, BO3 8001, RAG 8010, Packing 8020, frontend 8443.
- Session Supabase valide et au moins dix vêtements dans le compte de démo.
- Une question styliste, une tenue et un voyage déjà testés avant le jury.
- `DressMe_Presentation_FR.pdf` ouvert en secours.
- `A2A_TOKEN` exporté si vous prévoyez `a2a_discover.py --ask`.

Si vous n'avez que 8 minutes

Gardez l'ouverture, l'architecture, la slide plan de contrôle, la slide A2A et la conclusion. Sautez la vision, les entrées/sorties, la qualité et la démo ; dites que le PDF les contient. Énoncez les quatre justifications en une phrase chacune.

Dernière phrase avant les questions

« Tout ce que vous avez vu est soit une vraie ligne en base, soit une vraie réponse d'API, soit un calcul déterministe. Posez-moi vos questions sur n'importe quelle couche. »